StudyBuddy: Phase II System Design and Initial Implementation

Course: TCSS 460: Client/Server Programming for Internet Applications (Autumn 2026)
Group: Team 2 (Active Recall)
Members: Salima Hafurova, Jason Dinh, David Mwaura
Date: November 2, 2026

Part A: Project Scope and Phase I Revisions

Project Statement and Intended Users: University students that are taking detail-heavy courses (such as CS, Microbiology, Biochemistry, etc.) frequently struggle to convert large volumes of lecture notes into active recall flashcards while managing their study time. Traditional flashcard tools require manual card creation and separate time-tracking apps. Our app StudyBuddy supports college students and self-directed learners by integrating a 25-minute Pomodoro timer, an interactive active recall flashcard dashboard with keyboard shortcuts, per-deck mastery tracking (-5 to 5), and backend flashcard generation from raw study notes.

Phase I Grader Feedback and Revisions:

- **Feedback 1:** *"Make the agentic part concrete: name the function the AI calls to read card mastery scores."*

  Revisions Made: We defined the server-side tool function getLowMasteryCards(userId, deckId) paired with updateCardMastery(cardId, masteryScore) on our Node.js/Express backend using the Google Gemini Function Calling API. Rather than acting only as a passive text generator, the Gemini agent calls getLowMasteryCards(userId, deckId) to query flashcards from our database where mastery_score < 0 (on our -5 to 5 scale) and will generate targeted remedial cards for weak concepts.

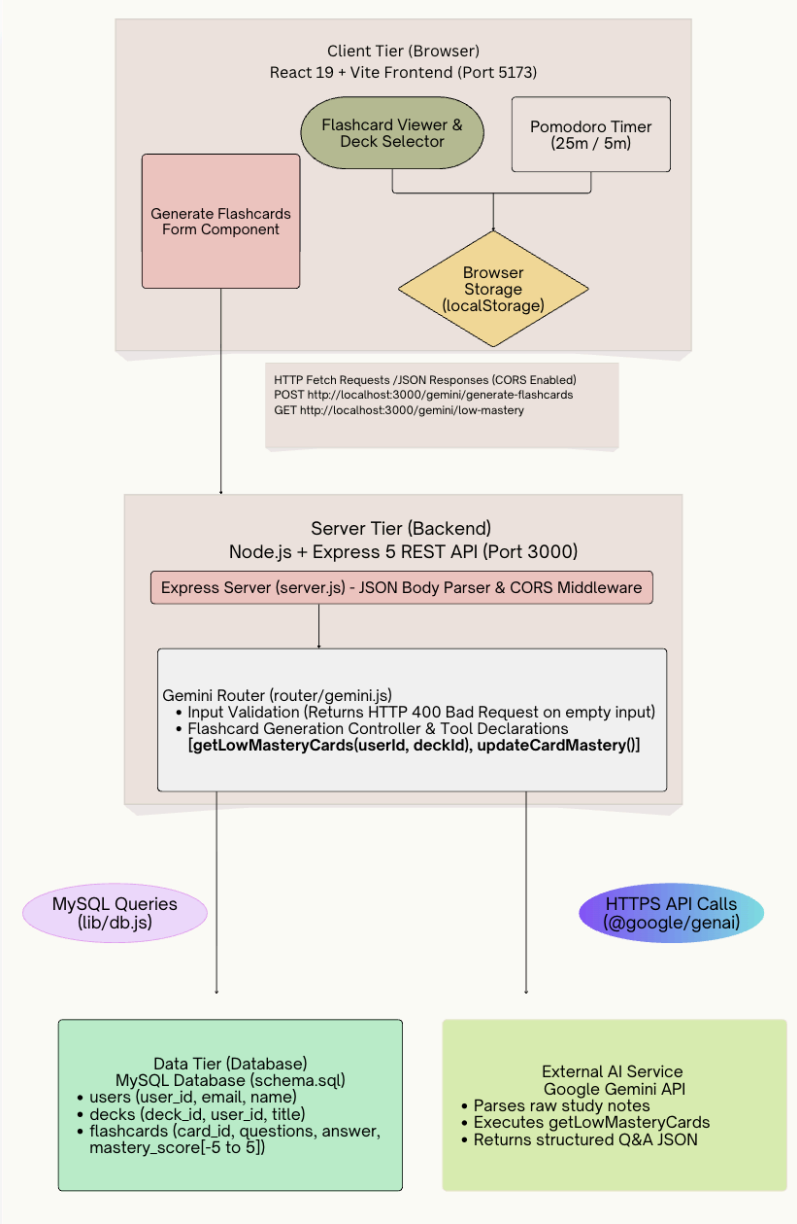
- **Feedback 2:** Remove all template prompt instructions from our documentation deliverables

Essential MVP Features vs. Optional Features

Feature	Priority	Description
Interactive Flashcard Viewer & Decks	Core (MVP)	Allows users to select preset or generated decks, flip cards (clip or Spacebar), and navigate questions.
Mastery Scoring (-5 to 5) & Persistent State	Core (MVP)	Tracks decks mastery via grading buttons/keys (1 and 2) and persists session state across page refreshes using localStorage and MySQL.
Pomodoro Study (25m) & Rest (5m) Timer.	Core (MVP)	Provides focus and reset timers that preserve running/paused countdown states across browser refreshes.
Express Flashcard Generation Web API	Core (MVP)	Validates pasted study notes (returning HTTP 400 if empty) and returns structured JSON flashcard arrays.
Agentic Review (getLowMasteryCards)	Core (Phase III)	Gemini tool call that reads low-mastery cards from MySQL and generates remedial flashcards.
Google OAuth 2.0 Login	Optional	Authenticates individual student accounts to sync flashcard decks across devices.

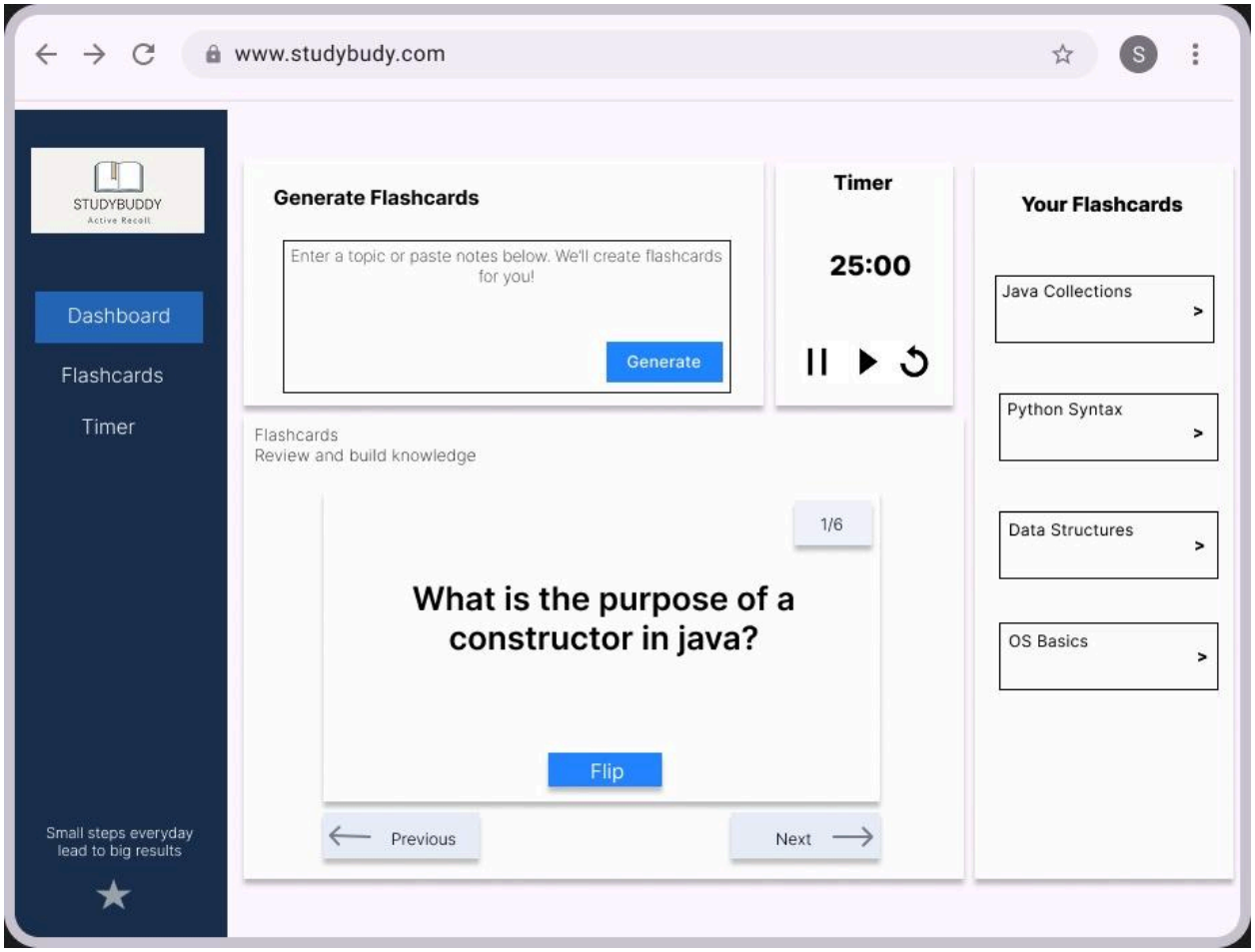
Part B: System Architecture and Frontend Design

B.1 - High-Level System Architecture Diagram



Component Responsibilities & Interactions: The React frontend (port 5173) manages user interactions, keyboard shortcuts, and local state persistence via localStorage. When a user submits study notes in the Generate Flashcards form, React sends an HTTP POST request to the Express backend (port 3000). Express validates the payload (returning HTTP 400 Bad Request if empty), processes the request via the Gemini router, interacts with the MySQL database pool (db.js), and returns structured JSON flashcards for React to render.

B.2 - Frontend Wireframe and User Interface



UI Elements: (1) Collapsible Frosted-Glass Sidebar for switching between Dashboard, Fullscreen Flashcards, and Fullscreen Timer views; (2) Generate Flashcards form with live backend validation feedback; (3) Pomodoro Timer with Study (25m) and Rest (5m) modes; (4) Interactive Flashcard Card supporting Spacebar flip, Arrow navigation, and number key press 1 (got the card correct) / 2 (got the card incorrect) keystroke grading; and (5) Decks selector list.

Part C: Preliminary Data Design

Primary Data & Storage Approach: Our application uses a relational MySQL database (defined in backend/schema.sql and connected via backend/lib/db.js) with three core tables: users, decks, and flashcards. During Phase II, client-side localStorage provides immediate session persistence across page refreshes.

Sample Data Structure (JSON):

```
{
   "deck_id": 3,
   "user_id": 1,
   "title": "Client/Server Programming",
   "mastery_score": 0,
   "cards": [
   {
       "card_id": 101,
       "question": "What is the primary communication pattern used in
client/server web apps?",
       "answer": "Stateless request/response pattern over HTTP; client
initiates a request & server processes it to send back a response.",
       "mastery_score": 0
   }
]
}
```

Data Operations: Create inserts newly generated decks and cards via POST /gemini/generate-flashcards; Retrieve loads decks and queries low-mastery cards (mastery_score < 0) via GET /gemini/low-mastery; Modify increments or decrements mastery scores between -5 and 5.

Part D: Web API Design

D.1 - Planned Web API Operations

HTTP Method	Endpoint	Functionality
POST	/gemini/generate-flashcards	(**Implemented in Phase II**) Validates study notes (HTTP 400 if empty) and returns generated JSON flashcards.
GET	/gemini/low-mastery	(**Implemented in Phase II)** Retrieves flashcards with mastery scores below 0 for weak-topic review.
GET	/api/decks	Retrieves all saved flashcard decks and mastery scores for the user from MySQL.
PATCH	/api/cards/:id/mastery	Updates a deck or card mastery score (-5 to 5) in the MySQL database.

POST	/api/agent/review	Invokes Gemini tool getLowMasteryCards(userId, deckId) to generate remedial flashcards.

D.3 - External APIs and Security

StudyBuddy integrates with the Google Gemini API (@google/genai) on the Express server to convert raw study notes into structured Q&A JSON cards and execute the getLowMasteryCards function declaration. To protect credentials, GEMINI_API_KEY is stored strictly in backend/.env (excluded via .gitignore) and is never exposed in frontend code.

Team Contributions (Phase II)

- **Jason Dinh**: Set up the GitHub repository, organized the backend directory structure, and implemented backend/server.js, backend/lib/db.js, and backend/routers/gemini.js.
- **David Mwaura**: Designed the UI wireframe for the dashboard, flashcard viewer, Pomodoro timer, and all the rest of the UI components for the side panel. Revised (App.jsx, and App.css) UI components to match layout concepts.
- **Salima Hafurova**: Implemented the React frontend (App.jsx, App.css), localStorage persistence, per-deck mastery scoring and keyboard shortcuts, MySQL schema (schema.sql), frontend-to-backend fetch() integration with HTTP 400 validation, and Phase II documentation.